Le dîner des philosophes en microservices

Du comportement naïf à l'exclusion mutuelle et à l'interblocage

Projet pédagogique – Master Informatique

Principe général

On considère N philosophes réactifs, chacun exécutant indépendamment une boucle infinie.

Étape	État du philosophe
1	**Penser** : le philosophe réfléchit.
2	**Avoir faim** : il souhaite manger.
3	**Manger** : il mange pendant un certain temps.
4	**Reprendre la pensée** : il retourne à l'état de réflexion.

Penser ⟶ Avoir faim ⟶ Manger ⟶ Penser

1 Philosophes gâtés

- On considère N instances indépendantes de `philosopher-service`.
  - Les durées de réflexion et de repas sont aléatoires.
  - Aucune ressource n'est partagée entre les philosophes.
  - Les philosophes peuvent donc manger simultanément.
- Un `monitor-service` reçoit les événements produits par les philosophes :

  `HUNGRY`, `EATING`, `DONE`.

  ‘
  - Chaque événement émis par un philosophe est associé à une horloge logique locale à ce philosophe, permettant au moniteur de reconstruire l'historique des événements.
  - Le moniteur fournit un tableau de bord permettant d'observer l'évolution des philosophes.

2 Philosophes placés sous la supervision d'un superviseur

- Dans cette version, on suppose qu'un philosophe qui souhaite manger interroge le `supervisor-service` pour savoir si une fourchette est disponible.
  - Si le superviseur répond positivement, le philosophe demande cette fourchette.
  - Une fois la première fourchette obtenue, le philosophe interroge à nouveau le superviseur pour obtenir la deuxième.

- Le `supervisor-service` ne réalise aucun contrôle global de cohérence : il ne décide pas si le philosophe peut manger et ne vérifie pas qu'il possède déjà l'autre fourchette.
- Son rôle est uniquement de :
  - attribuer une fourchette lorsqu'elle est disponible ;
  - accepter sa libération lorsqu'un philosophe la restitue.

- Donner une implémentation naïve de cette solution, puis recenser ses failles à l'aide d'un jeu de tests.

3 Section critique, solution avec verrou

3.1 Sans verrou

- Jusqu'à présent, le superviseur reçoit les demandes des philosophes et gère l'état des fourchettes. On suppose qu'un philosophe demande les deux fourchettes en même temps au superviseur.
- Dans un serveur web, plusieurs requêtes peuvent être traitées simultanément par plusieurs threads. Par exemple :

```
P1 ---- GET /fork/1 ----> Thread T1 ----\
                                        \
P2 ---- GET /fork/2 ----> Thread T2 ------> Superviseur
                                        /
P3 ---- GET /fork/3 ----> Thread T3 ----/
```

- Plusieurs philosophes peuvent donc effectuer des demandes simultanément. La partie du code qui modifie l'état partagé des fourchettes doit s'exécuter de manière **atomique**. Cette propriété n'est pas nécessairement garantie dans cette première version.

3.2 Avec verrou

- Une **section critique** est une partie du programme qui accède à une ressource partagée et qui ne doit pas être exécutée simultanément par plusieurs threads.
- Dans notre problème, un philosophe qui veut manger doit obtenir deux fourchettes :

$$F_i \quad \text{et} \quad F_{i+1}$$

- Le superviseur doit effectuer l'opération suivante :

```
vérifier F_i et F_{i+1}
          |
          v
les deux sont-elles libres ?
          |
       oui|       non
          |         |
          v         v
 réserver les deux  refuser
```

- Modifiez votre implémentation pour verrouiller les sections critiques liées à l'accès aux fourchettes.

3.3 Protocole distribué par jetons

- On considère N microservices philosophes. Chaque philosophe peut demander deux fourchettes quelconques, sans contrainte de voisinage. Les microservices philosophes communiquant par échange de messages. Chaque philosophe P_i gère une fourchette F_i. Il n'existe pas de microservice dédié aux fourchettes. ‘
- Chaque fourchette est associée à un jeton unique représentant le droit exclusif de l'utiliser. Les jetons circulent entre les microservices philosophes.
- Lorsqu'un philosophe souhaite manger, il diffuse une demande à l'ensemble des autres philosophes. Chaque demande contient l'identifiant du philosophe demandeur ainsi qu'un horodatage logique.
- Chaque philosophe reçoit les demandes, les mémorise et peut être amené à transférer son jeton en fonction du protocole distribué que vous devez concevoir.
- Un philosophe ne peut commencer à manger que lorsqu'il dispose des deux jetons correspondant aux deux fourchettes qui lui sont attribuées par le protocole. À la fin du repas, il libère les fourchettes et les jetons peuvent être transmis à d'autres philosophes.
- Concevoir le protocole distribué permettant de traiter les demandes concurrentes et de transférer les jetons.
- Définir la règle d'attribution automatique des deux fourchettes à un philosophe demandeur.
- Implémenter la diffusion, la réception et la mémorisation des demandes horodatées.
- Gérer les demandes simultanées et les éventuels délais d'attente ou de retransmission des demandes.
- Garantir qu'une même fourchette ne peut pas être utilisée simultanément par plusieurs philosophes.
- Le protocole doit garantir l'équité entre les philosophes, en évitant qu'un philosophe soit privé de l'accès aux fourchettes pendant une période excessivement longue par rapport aux autres.
- Étudier les risques d'interblocage et de famine. Proposer et implémenter les mécanismes nécessaires pour les éviter.
- Tester le système avec plusieurs philosophes demandant à manger simultanément et analyser les traces d'exécution.

Contraintes

- Chaque philosophe est un microservice Spring Boot.
- Les philosophes communiquent par échange de messages HTTP.
- Aucun superviseur central ne décide de l'attribution des fourchettes.
- Le protocole doit rester correct lorsque plusieurs demandes sont traitées concurremment.

3.4 Comparaison avec un outil de coordination distribuée

Après avoir conçu et implémenté votre propre protocole distribué d'exclusion mutuelle pour le problème des philosophes, utilisez Apache ZooKeeper ou un autre outil de coordination distribuée afin de mettre en place une seconde solution reposant sur un mécanisme de verrouillage distribué. Comparer cette solution avecla précédente.

4 Tests unitaires et tests d'intégration

Vous devez mettre en place une stratégie de tests permettant de vérifier le bon fonctionnement des microservices philosophes et la correction du protocole distribué.

1. **Tests unitaires :** utiliser JUnit 5 et Mockito pour tester indépendamment les principales fonctionnalités d'un philosophe : traitement des demandes horodatées, gestion des jetons, attribution et libération des fourchettes, ainsi que traitement des erreurs.
2. **Tests d'intégration :** vérifier le fonctionnement conjoint de plusieurs microservices philosophes, notamment la diffusion des demandes, le transfert des jetons et la coordination des accès aux fourchettes. Utiliser les outils de test Spring Boot adaptés.
3. **Tests de concurrence :** déclencher plusieurs demandes simultanées et vérifier qu'une même fourchette n'est jamais attribuée à plusieurs philosophes en même temps.
4. **Tests de résilience :** simuler des délais de réponse, des indisponibilités et des interruptions de communication. Vérifier le comportement du système en présence de ces défaillances.
5. **Analyse des résultats :** distinguer les propriétés effectivement vérifiées par les tests de celles qui nécessiteraient une vérification formelle. Présenter les résultats et les éventuelles limites de la stratégie de test.

5 Architecture Master–Worker : calcul distribué sur un graphe

- On souhaite étendre le projet en adoptant une architecture *Master–Worker*. Un philosophe joue le rôle de *Master*. Il dispose d'un grand graphe (*Big Graph*) dont il souhaite calculer le degré de chaque nœud. Il répartit le graphe en plusieurs sous-graphes et distribue les tâches de calcul aux autres philosophes, qui jouent le rôle de *Workers*.

    1. Concevoir une architecture Master–Worker dans laquelle le Master répartit les données du graphe et les tâches de calcul entre les Workers.
    2. Proposer une stratégie de partitionnement du graphe et préciser les données transmises à chaque Worker.
    3. Chaque Worker calcule le degré des nœuds qui lui sont attribués et renvoie ses résultats au Master.
    4. Le Master collecte les résultats partiels, les rassemble et produit le résultat final. Préciser comment éviter les doubles comptages lorsque des arêtes relient des nœuds appartenant à différentes partitions.
    5. Utiliser les communications entre microservices pour distribuer les tâches et transmettre les résultats.
    6. Étudier le comportement du système lorsqu'un Worker devient indisponible ou renvoie un résultat tardivement. Proposer une solution permettant de relancer les tâches concernées sans compromettre la correction du résultat final.
    7. Évaluer les performances de l'approche en faisant varier le nombre de Workers et la taille du graphe. Comparer les temps d'exécution et discuter les limites du parallélisme.

- On souhaite paralléliser le problème d'accessibilité dans un graphe.

Rappel : Problème d'accessibilité dans un graphe

Soit $G = (V, E)$ un graphe orienté, où V désigne l'ensemble des nœuds et E l'ensemble des arcs. Étant donnés deux nœuds $s, t \in V$, le problème d'accessibilité consiste à déterminer s'il existe un chemin de s vers t.

Formellement, il s'agit de déterminer s'il existe un entier $k \geq 0$ et une séquence de nœuds $v_0, v_1, \ldots, v_k$ tels que :

$$v_0 = s, \qquad v_k = t,$$

et

$$\forall i \in \{0, \ldots, k-1\}, \quad (v_i, v_{i+1}) \in E.$$

Si un tel chemin existe, on dit que t est accessible depuis s. Sinon, t n'est pas accessible depuis s.

6 Approche DevOps

Intégrer au projet une chaîne de déploiement et d'exploitation basée sur Docker pour la conteneurisation des microservices, Kafka pour la communication asynchrone entre les services et Kubernetes pour l'orchestration des conteneurs. Mettre en place l'automatisation du processus de construction, de test et de déploiement à l'aide d'une chaîne CI/CD (par exemple, GitHub Actions), ainsi que des mécanismes de supervision, de journalisation et de gestion des pannes, afin de garantir la fiabilité, la résilience et la reproductibilité du déploiement.

7 Livrables attendus

Le rendu du projet doit comprendre les éléments suivants :

- **Code source :** dépôt Git contenant le code de l'ensemble des microservices, les fichiers de configuration et les instructions nécessaires à l'installation et à l'exécution de l'application.
- **Rapport technique :** document présentant l'architecture retenue, les protocoles conçus, les choix d'implémentation, les difficultés rencontrées et les solutions apportées.
- **Tests :** code des tests unitaires et d'intégration, scénarios de test de concurrence et de panne, ainsi que les résultats obtenus.

8 Ressources documentaires et techniques

Les ressources suivantes vous aideront à comprendre les concepts, à réaliser le projet et à approfondir les solutions étudiées. Elles ne remplacent pas le travail de conception, d'implémentation et de validation qui vous est demandé.

- **Spring Boot – documentation officielle :**
  `https://docs.spring.io/spring-boot/`
- **Apache ZooKeeper – documentation officielle :**
  `https://zookeeper.apache.org/`
- **Verrous distribués avec ZooKeeper :**
  `https://zookeeper.apache.org/doc/current/recipes.html`
- **Apache Curator – recettes de coordination distribuée :**
  `https://curator.apache.org/docs/recipes/`
- **Tests d'intégration avec Testcontainers et Spring Boot :**
  `https://docs.spring.io/spring-boot/reference/testing/testcontainers.html`

- **Resilience4j – gestion de la résilience :**
  `https://github.com/resilience4j/resilience4j`
- **GitHub – exemple du problème des philosophes :**
  `https://github.com/chutrunganh/Dining-philosophers-problem`
- **Cours et diapositives sur les systèmes distribués :**
  `https://www.cs.uic.edu/~ajayk/DCS-Book`
- **Notes de cours sur l'exclusion mutuelle distribuée :**
  `https://people.cs.rutgers.edu/~pxk/classes/417/notes/mutex.html`

8.1 Supports de cours à consulter

Pour préparer les différentes étapes du projet, vous pourrez rechercher ou consulter des supports de cours portant sur :

- les microservices, les API REST et Spring Boot ;
- les processus concurrents, les sections critiques, les moniteurs et les verrous ;
- les algorithmes d'exclusion mutuelle distribuée ;
- les horloges logiques et l'ordonnancement des événements ;
- les interblocages, la famine et les propriétés d'équité ;
- les tests unitaires, les tests d'intégration et l'injection de pannes ;
- les verrous distribués, ZooKeeper et les mécanismes de coordination ;
- la résilience, les timeouts et la tolérance aux pannes.

8.2 Utilisation responsable de l'intelligence artificielle

Sachez utiliser l'IA !

L'intelligence artificielle peut vous aider à comprendre un concept, à explorer une solution ou à diagnostiquer une erreur. Cependant, elle ne doit pas remplacer votre propre réflexion.

Un mauvais usage de l'IA limite la connexion entre vos neurones : à force de déléguer votre réflexion, vous risquez de ne plus développer les compétences que ce projet est précisément conçu pour vous apprendre.

Toute solution proposée par une IA doit être comprise, justifiée, testée et validée par vos soins. Vous devez être capables d'expliquer vos choix, de reproduire vos résultats et de défendre votre implémentation sans dépendre de l'outil.

L'objectif n'est pas de vous interdire l'IA, mais de vous apprendre à l'utiliser comme un assistant plutôt que comme un substitut à votre raisonnement.